EP08 Pattern Recognition – Machine Learning

2nd Assignment: Neural Networks for Image Classification

Assignment type: Individual

Deadline: Wednesday 17/06/2026, 23:55 (No extension will be granted)

Submission method: Exclusively via eclass (assignments sent by email will not be accepted)

Total points: 100 (40 points Notebook + 60 points oral examination)

(20% of the final course grade)

The oral examination will take place on Monday 29/06/2026.

Objective

In this assignment you will train neural networks, Feedforward and Convolutional, for the stylistic classification of paintings into four art movements, using a subset of WikiArt. You will develop the full cycle of a deep learning system in PyTorch, which includes data preparation, architecture design, training loop, model selection via a validation set, optimization with modern techniques (BatchNorm, regularization, optimizers, data augmentation, LR scheduling), transfer learning from a pretrained ResNet18, and visualization of the internal representations.

The notebook is your preparation and accounts for 40% of the final grade. The main assessment takes place in the oral examination (60%), where you will be questioned on your code and your results as submitted on eclass. If the results (models, plots, metrics) are not available or you cannot explain your choices and your results, the grade of the assignment will be correspondingly low, regardless of the quality of the code.

Context

Classifying paintings into art movements was for decades the exclusive work of experts. Today museums maintain millions of digitized works, and automatic stylistic classification of art is an active field at the intersection of Computer Vision and the Digital Humanities. The question is not only whether neural networks can classify paintings, but also what exactly they "see" when they do so.

You will work with four movements, each with a distinct visual identity:

- Baroque (1600–1750): dramatic lighting, dark background, realistic rendering. Painters such as: Caravaggio, Rembrandt, Vermeer.
- Impressionism (1860–1900): pastel palette, visible loose brushstrokes, outdoor scenes. Painters such as: Monet, Renoir, Pissarro.
- Cubism (1907–1920): geometric fragmentation, palette of ochres and greys, multiple viewpoints in a single work. Painters such as: Picasso, Braque.
- Abstract Expressionism (1940–1960): non-representational works, gestural brushstrokes or color fields, absence of a recognizable subject. Painters such as: Pollock, Rothko, de Kooning.

Answer structure

The assignment consists of 10 questions (Q0.1–Q5.1) that are answered within the notebook, in a markdown cell immediately after the corresponding code, with a title exactly the same as that of the assignment (e.g. ## *Q1.1 Based on the loss curves...*). Each answer must be short (5–10 sentences), answer the question directly, and be supported by reference to specific results of the notebook (metrics, plots, tables).

An answer that remains at a general theoretical level, without reference to the results of your notebook, is considered incomplete.

0. Data

0.1 Download and preparation

WikiArt is the most widely used dataset for computational analysis of art, with approximately 80,000 digitized works in 27 movements. The full dataset requires about 30GB, so for this assignment you are given a ready, preprocessed subset of four classes at low resolution.

Download the file `wikiart_hw2.npz.zip` from eclass (Documents → Data for the 2nd Assignment 2026) and unzip it to extract `wikiart_hw2.npz` (~15MB). It is a compressed numpy archive containing:

- `X`: array of shape (5400, 32, 32, 3), of type uint8, with RGB images in channels-last format and pixel values in the range 0–255. The images have already been center-cropped on the shorter side and resized to 32×32.
- `y`: array of shape (5400,), of type int64, with labels 0–3.
- `class_names`: array with the class names in the order of the labels.

The dataset is balanced (1,350 samples per class) and already shuffled with `random_state=42`. The mapping of labels to movements is: 0 → Baroque, 1 → Impressionism, 2 → Cubism, 3 → Abstract_Expressionism.

- Load the file and convert the data into PyTorch tensors. X as float32, normalized to the range [0, 1] (division by 255), and in channels-first format [N, 3, 32, 32] (hint: `torch.permute`). y as int64.
- Create balanced splits with `train_test_split` (from `sklearn.model_selection`) using `stratify=y` and `random_state=42`. Target: 800 train, 200 val, 350 test per class (in total 3200 / 800 / 1400). Since `train_test_split` produces two sets each time, call it twice in succession.
- Create PyTorch DataLoaders with `batch_size=16`, `shuffle=True` for train/val and `shuffle=False` for test.
- Print split statistics and confirm that each set contains the expected number of samples per class.
- Visualize two random images per class in a 2×4 layout (rows: examples, columns: classes).

Q0.1 (3 pts)

From the visualization, which visual features seem to differentiate the four movements from each other the most (color palette, texture, composition, presence of a subject)? Which pairs of classes do you expect to be easiest for a classifier to confuse, and why? If a human saw the same images resized to 32×32, which movement would be the hardest to distinguish?

1. Feedforward Neural Network on raw pixels

Before we use an architecture designed for images, let us see what a simple FFN does when it receives the image as a flat vector of 3,072 values (32×32×3 flattened). It is your first PyTorch network and it will serve as a baseline for the CNN of Question 2.

1.1 Architecture and training

Define, as a subclass of `nn.Module`, an FFN with input 3,072 (the flattened image), three fully connected layers with 128, 32, and 4 neurons respectively, and ReLU activation between the layers (but not after the last linear layer, which leads to the output).

Then define two functions. `train_one_epoch` runs one training epoch (forward pass, loss computation, backward pass, parameter update) and returns the mean train loss of the epoch. `evaluate` passes all batches of a dataloader through the model in eval mode without updating the weights, and returns loss, accuracy, macro-averaged F1, and confusion matrix.

Train the network with the Adam optimizer (`lr=0.001`), loss function `nn.CrossEntropyLoss`, and 30 epochs. At each epoch evaluate on the validation set and keep the `state_dict` of the model with the best F1 macro performance on validation. At the end, load this `state_dict` and evaluate on the test set.

Train the network on both CPU and GPU (with exactly the same hyperparameters and number of epochs), and print the execution times. Present two plots: (a) train and validation loss as a function of the epoch in the same figure, (b) validation F1 macro as a function of the epoch with a vertical line marking the epoch at which the best model was selected (for best model selection the validation F1 macro is used, not accuracy).

Q1.1 (5 pts)

Based on the loss curves and the validation F1 curve, do you observe overfitting, underfitting, or balance? At which epoch was the best model selected with the validation F1 as criterion, and at which would it have been selected if you used the train loss as criterion? Why is selection based on the validation set critical? Was the GPU speedup relative to training on CPU expected for a network of this size?

Q1.2 (3 pts)

Present the accuracy, F1 macro, and confusion matrix of the best model on the test set. Which movements are confused with each other the most? Reshape the weights of the first linear layer from shape [128, 3072] to [128, 3, 32, 32], normalize each "filter" to [0, 1] (min-max per filter), and visualize 16 of them as RGB images in a 4×4 layout. Discuss why

the flat-pixel FFN struggles despite the presence of ReLU: what does each neuron of the first layer "see", and what kind of image structure can this architecture not exploit?

2. Convolutional Neural Network

In this section you will use the images as tensors (not flattened) and you will gradually see the effect of each component of a CNN on training and performance.

2.1 Incremental construction

Define a CNN with a sequence of four convolutional layers (kernel size 5) that implements the channel sequence 3, 16, 32, 64, 128 (where the initial 3 corresponds to the RGB input channels). The output of the last convolutional layer enters a fully-connected head of four layers with x, 1024, 256, 32, 4 neurons (where x is the output dimension of the flatten, which may differ between the versions below).

You will train three versions of the network, each one adding a component:

- §2.1 baseline: no padding, no pooling, no activations.
- §2.2 with padding and pooling: `padding=2` in every convolution and `MaxPool2d(kernel_size=2)` after every conv, no activation yet.
- §2.3 with activations: add ReLU after every convolution (**before the pooling**) and before every linear layer of the FC head, except the last one that leads to the output.

Train each version with the settings of Question 1 (Adam, lr=0.001, 30 epochs) and measure the total number of parameters per version (`sum(p.numel() for p in model.parameters())`). Train §2.3 on both CPU and GPU.

Q2.1 (4 pts)

Fill in the table:

Version	# params	Val Acc	Test Acc	F1 macro
§2.1 (no pad/ pool/activ)				
§2.2 (with pad and pool)				
§2.3 (with ReLU)				

Which change had the greatest effect on performance and which on the number of parameters? If §2.1 achieves low or near-random performance, explain technically why this happens, beyond the general observation that activations are missing.

Q2.2 (3 pts)

Compare the final CNN (§2.3) with the FFN of Question 1 on two metrics: the number of parameters and the test F1 macro. Does the CNN have more or fewer parameters than the FFN? How different is the performance? Discuss why the number of parameters on its own does not predict performance, and what role the structure of the parameters plays

here (how they are organized and what prior knowledge about images they encode). Also compare the confusion matrices: in which movements does the CNN substantially improve? How does the CPU-vs-GPU speedup of the CNN compare with that of the FFN, and why?

Q2.3 (3 pts)

Scenario: A museum is designing a system for automatic stylistic classification of a digitized collection of 2,000,000 works. You are proposed to deepen the network (ten convolutional layers instead of four), with an expected benefit of +2% in test accuracy but several times the computational cost. Base your answer on the CPU/GPU times you measured (§1 and §2.3) and on the train–val behavior you observed. In which cases is this choice reasonable and in which not? Consider the deployment context (real-time versus batch processing), energy cost, and the need for interpretability by curators.

3. Performance Improvement

In this section you will apply modern CNN improvement techniques: reproducibility, alternative optimizers, learning rate scheduling, batch normalization, data augmentation, and regularization. Each step starts from the architecture of §2.3.

3.1 Reproducibility, optimizers, LR schedule, BatchNorm

To reliably compare the techniques that follow, training must be fully reproducible. Otherwise, random variations in weight initialization or in the order of the batches may mimic the benefits of a technique.

Consult the PyTorch documentation for reproducibility and apply the appropriate settings: seed for random, numpy, torch, and `torch.cuda.manual_seed_all`, as well as `torch.backends.cudnn.deterministic = True` and `torch.backends.cudnn.benchmark = False`. Run the training of §2.3 twice and confirm that you get exactly the same train loss at every epoch and the same performance on the test set.

Then train §2.3 with five different optimizers in order to see whether any of them outperforms Adam (which is already your baseline from Question 2). Consult the PyTorch optimizers documentation:

- SGD with lr=0.01
- SGD with momentum 0.9 and lr=0.01
- Adam with lr=0.001 (baseline)
- RMSprop with lr=0.001
- AdamW with lr=0.001 and weight_decay=0.01

To the best optimizer resulting from the comparison, add `CosineAnnealingLR` as a learning rate scheduler with `T_max=30` (equal to the number of epochs), and compare with the version without a scheduler. Finally, insert `nn.BatchNorm2d` before every ReLU in all convolutional layers, and train with the best optimizer and the scheduler.

Q3.1 (6 pts)

Fill in the optimizers table (without LR schedule and without BatchNorm):

Optimizer	Val Acc	Test Acc	F1 macro	Best epoch
SGD				
SGD with momentum				
Adam				
RMSprop				
AdamW				

Which optimizer prevailed in performance and in convergence speed (best epoch)? Discuss the observed difference between SGD and adaptive optimizers (Adam, RMSprop, AdamW). How was training affected by the cosine LR schedule and by the addition of BatchNorm? Describe intuitively what an LR schedule does during training.

3.2 Data augmentation and regularization

In §3.1 you probably observed a significant divergence between train and validation loss, which indicates overfitting. In this subsection you will try to reduce it in two complementary ways: from the data side with augmentation, and from the parameter side with regularization.

The baseline architecture is the final CNN of §3.1 (§2.3 with BatchNorm, the best optimizer, and cosine LR schedule). You will use it in all the experiments that follow, trained for 60 epochs (instead of 30), adjusting the `T_max` of the scheduler accordingly.

For data augmentation apply to every training image (not to val/test) through `torchvision.transforms`:

- `RandomHorizontalFlip(p=0.5)`
- `RandomCrop(32, padding=4)`
- `ColorJitter(brightness=0.1, contrast=0.1)`

Some transforms that are often applied to natural images (e.g. `RandomRotation`, `RandomVerticalFlip`, or extreme `ColorJitter`) are semantically wrong for paintings. Think about why, before applying any of them.

For regularization apply weight decay=1e-4 in the optimizer and `Dropout(p=0.3)` before every linear layer of the FC head.

Try four configurations: baseline (no additional augmentation), augmentation only, regularization only, and finally both together. For each configuration, plot the train and validation loss curves in the same figure.

Q3.2 (6 pts)

Fill in the table:

Configuration	Best epoch	Val F1	Test F1	Train–Val gap
None (§3.1 baseline, 60 epochs)				
With data augmentation				
With weight decay and dropout				
With augmentation and regularization				

In which configuration is the train–val gap (val_loss − train_loss) reduced the most, and in which does the performance on the test set increase? Do these two coincide? What do you conclude about the relationship between good generalization and the absence of overfitting? Which transforms did you decide would be wrong for paintings, and why?

4. Transfer Learning with a Pretrained ResNet18

You have reached the limits of the performance you can achieve by training from scratch on 3,200 paintings. In this section you will use a pretrained ResNet18, an 18-layer network with about 11 million parameters, trained on 1.2 million ImageNet images (cats, dogs, cars, everyday objects, but not paintings). You will check whether what it learned in such a different domain transfers to art classification.

4.1 Linear probe on a frozen ResNet18

Load the pretrained model and modify it as follows:

- Load `torchvision.models.resnet18(weights="IMAGENET1K_V1")`.
- Freeze all its weights so that they are not updated during training (`for p in model.parameters(): p.requires_grad = False`).
- Replace the last fully-connected layer (`model.fc`, with 1,000 outputs for the ImageNet classes) with a new `nn.Linear(512, 4)` that will be trained from scratch.
- Resize the images to 224×224 (the input size that ResNet18 expects). You can upscale the 32×32 tensors with `torch.nn.functional.interpolate(images, size=224, mode='bilinear', align_corners=False)`.
- Normalize the images with the ImageNet statistics: `mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`.

Train only the new fc layer with Adam (lr=0.001) for 10 epochs, and evaluate on the test set.

Note: ResNet18 has been trained on 224×224 images. Upscaling from 32×32 introduces artifacts that may affect performance.

Q4.1 (4 pts)

Fill in the final comparison table:

Model	Test Acc	F1 macro	Trainable params	Training time
FFN flat pixels (§1)				
Best CNN (§3.2)				
Frozen ResNet18 with linear probe (§4.1)				

Compare the performance of the frozen ResNet18 with the CNN you trained yourself from scratch. If ResNet18 performs better, even though it has never seen a work of art during its pretraining, how is this explained and what does it tell you about how a network learns, from ImageNet, representations that are generally useful for vision, even in a completely different domain such as art? If it performs worse, which factors of this specific implementation (e.g. the original 32×32 resolution and the upscaling to 224×224, or the restriction to a linear probe) may be responsible?

5. What the Networks Learned

You now have two trained networks: the CNN you built yourself (§3.2) and the frozen ResNet18 (§4.1). In this section you will compare what each one has learned in its first convolutional layer.

5.1 Comparison of the first conv layers

Extract the weights of the first convolutional layer from each network and visualize them as small RGB images. From your own CNN, the weight tensor has shape $[16, 3, 5, 5]$: 16 filters, each 5×5 over 3 channels. Normalize each filter to $[0, 1]$ (min-max per filter) and visualize the 16 filters in a 4×4 layout. From ResNet18, the tensor `model.conv1.weight` has shape $[64, 3, 7, 7]$; select the first 16 filters and visualize them in the same way.

Then extract the activation maps of the first conv layer for a random sample of each class. Select one painting from each class from the test set, do a forward pass through each network, and store the activations of the first conv layer after the ReLU. Visualize the first 16 channels as heatmaps for each painting and each network.

Q5.1 (3 pts)

Compare the first 16 filters of your own CNN with those of ResNet18. In which network do the filters look clearer and more interpretable (recognizable edge detectors, color blobs, light–dark contrasts)? Is this expected, and why? From the activation maps, which filters are activated the most for each movement, and is there one that stands out clearly (e.g. a filter that lights up in dark regions for Baroque)? Connect your observations with what you wrote in Q0.1: what of the things you predicted as differentiating between movements does each network seem to have captured? Finally, contrast the convolutional filters of both networks with the weight images of the first FFN layer that you visualized in Q1.2: what do you observe when we move from flat-pixel weights to spatial convolutional filters?

Oral Examination (60 points)

Duration: 15 minutes per student. You will be questioned on the notebook and your results, and you will be asked to explain your choices and to answer new questions.

The oral examination is based on the notebook as submitted. If the results (models, plots, metrics) are not available or cannot be presented immediately, the assignment is considered insufficiently documented.

Oral examination grading scale: 55–60 in-depth understanding and ability to answer new questions, 40–54 adequate understanding and justification of the main choices, 20–39 superficial knowledge and difficulty with follow-up questions, 0–19 inability to explain.

Grading

Section	Points
0. Data	3
1. Feedforward Neural Network on raw pixels	8
2. Convolutional Neural Network	10
3. Performance Improvement	12
4. Transfer Learning with a Pretrained ResNet18	4
5. What the Networks Learned	3
Notebook Total	40
Oral Examination	60
Total	100

In case of technical problems

If you encounter a serious technical problem (CUDA out-of-memory, model non-convergence, unstable results between runs):

- Write in the notebook what you tried, what error you got, and what you did to solve it.
- If a model shows an error or does not train, explain what you think is wrong and what you would expect if it worked.
- A technical problem that is properly documented does not imply a grade reduction, provided that the rest of the assignment is complete.

Use of AI Tools

You may use AI tools (ChatGPT, Claude, Copilot, etc.) for debugging, PyTorch syntax questions, and clarification of concepts. You are not allowed to ask an AI to write the answers to the questions or to interpret results on your behalf. In the oral examination you

may be asked to explain parts of the code that appear to have been generated by an AI tool.

You must include a file `AI_declaration.md` in which you state which tools you used, for what purpose, and declare that you understand the code and the results.

Implementation and submission instructions

The assignment is implemented in a Jupyter notebook (Google Colab or an equivalent environment). Make sure that the notebook runs without errors from start to finish (Run All) before submission. Each question is answered in a markdown cell immediately after the corresponding code, with clear formatting and structure to facilitate reading and grading.

For reproducibility, add at the beginning of the notebook:

```
import numpy as np, random, torch
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

In all `train_test_split` calls and sklearn/PyTorch initializations that involve randomness, use `random_state=42` or equivalent.

A ZIP file named `AM_Surname.zip` (e.g. `1115XXXX_Papadopoulos.zip`), containing:

- `AM_Surname.ipynb`: notebook with code, results, and answers.
- `README.txt`: full name, student ID (AM), list of dependencies (pip install ...), estimated execution time on GPU.
- `AI_declaration.md`.

Do not include the file `wikiart_hw2.npz` in the ZIP. We assume that the grader has it locally. In your code assume that it is located in the same folder as the notebook.